NANYANG TECHNOLOGICAL UNIVERSITY

Multimodal Multi-Agent Triage Assistant for Hospital-at-Home Chronic Disease Monitoring

CA6117 · AI in Healthcare

Course Project Report

Group 14 — Individual Project

Lin, Jie-Rou

Matriculation Number — G2509197F

College of Computing and Data Science

October 2026

1. Introduction, Problem, and Background

1.1 Hospital-at-Home (H@H) as a data and workflow setting

Hospital-at-Home (H@H) moves hospital-level care from the acute ward into a patient's home. The course materials describe a virtual bed as a formal hospital admission in which care is delivered through home visits, teleconsultation, and remote physiological monitoring, while capacity is constrained by staff reach and monitoring rather than physical bed space [2]. They also emphasize that a virtual bed needs observability. Between scheduled touchpoints, escalation depends on what can be sensed remotely and on what the patient or caregiver notices and reports [2]. This makes information flow a central part of safe H@H operations.

The home environment is materially different from a ward. Observation can be less frequent, sensors may be applied or removed by patients or caregivers, connectivity and battery limitations can interrupt telemetry, and verification of an alert may require a phone or video call or a physical journey [2]. Therefore, the cost of both false positives and false negatives is amplified. A false positive may trigger avoidable clinician work or travel, while a missed deterioration signal can persist until the next scheduled contact. In this setting, scaling is not only about collecting more data. It is about enabling clinicians to review the right data, with the right context, at the right time.

This project focuses on patients with concurrent chronic conditions, specifically heart failure and type 2 diabetes. Such cases are useful for an agentic workflow because the same home-monitoring episode can contain physiological measurements, free-text symptom descriptions, diet-related context, and potentially conflicting interpretations across clinical domains. The project does not attempt to replace clinical judgment. Instead, it investigates whether an agentic system can structure multimodal information, retrieve relevant evidence, coordinate specialist reasoning, and prepare an auditable case for clinician review.

1.2 Clinical coordination burden and alert fatigue

Remote monitoring can increase observability, but it can also increase the number of signals that clinicians must interpret. A case study discussed in the course materials

illustrates that an apparently useful predictive model can still impose an impractical alert burden in deployment. The cited external validation required 109 alerts to be reviewed for each additional patient found [3,6]. The lesson for this project is broader than sepsis. Model performance alone is not system performance. An H@H triage tool must be designed around workload, attention, verification, and escalation, not only around the correctness of an isolated model output.

The original project proposal framed this problem as a need to synthesize structured physiological data with unstructured behavioural context so that care teams are not forced to manually reconcile each signal in isolation. That motivation is retained, but the final prototype narrows the action scope. Rather than drafting a patient intervention for automatic release, the implemented system is clinician-facing and draft-only. It structures the case, exposes evidence, identifies safety reasons, and stops for human review. This refinement reduces action authority while strengthening traceability and alignment with the course emphasis on bounded goals, explicit guardrails, stop conditions, and human oversight [1,4].

1.3 Intended users and workflow objective

The primary intended user of the final prototype is a clinician or clinical operations team responsible for reviewing H@H monitoring cases. Patients and caregivers are represented as upstream data sources rather than as direct recipients of autonomous AI recommendations. The clinician needs to answer a practical question. Given the available home-monitoring data and guideline evidence, is this case sufficiently complete and internally consistent to proceed to routine review, or should the workflow stop and be escalated because a safety, data-quality, dependency, or conflict condition has been detected?

The project objective is therefore to build a glass-box agentic triage workflow that makes the following elements visible and inspectable. These include patient input, planning and state transitions, deterministic tool calls, local guideline retrieval, specialist reasoning, safety checks, the final workflow disposition, clinician review actions, and a complete audit record. This objective directly matches the project brief, which requires visible agent workflow, grounding, human checkpoints, safety responses, audit logs, and evaluation artefacts [1].

1.4 Scope and non-goals

The prototype is intentionally scoped as decision support for workflow triage rather than autonomous clinical decision-making. It does not independently diagnose heart failure decompensation or diabetes complications, prescribe medication, set treatment plans, or communicate a clinical recommendation directly to the patient. In implementation terms, automated action authority is capped at a draft/pend level. The system can prepare information for review and can deterministically block continuation, but it cannot commit a high-impact patient-facing action.

The project also treats several thresholds as prototype policies rather than clinical rules. For example, data older than 24 hours are marked as stale for the purpose of demonstrating fail-closed workflow behaviour. This threshold is explicitly labelled in the interface as a prototype freshness policy, not a guideline-derived clinical threshold. Similarly, the heart-failure workflow calculates weight change but does not infer that a guideline weight-gain threshold has been met unless the necessary time interval is actually available. This distinction prevents the prototype from converting incomplete home data into unsupported clinical claims.

2. Agentic Solution / System Design

2.1 Overall architecture

The implemented system follows a multi-agent orchestration pattern. A central orchestrator coordinates a Heart Failure specialist agent and a Glycemic specialist agent. Each specialist combines deterministic tool outputs with domain-specific guideline retrieval and a structured reasoning component. Their outputs are not released directly. Instead, a deterministic Safety Engine evaluates input quality, security signals, multimodal consistency, dependency status, reasoning status, and cross-agent disagreement before the workflow is routed to clinician review or SAFE_STOP.

The user-facing layer is a React and TypeScript clinician dashboard, while the backend is implemented in FastAPI. The local RAG layer uses precomputed document chunks and dense embeddings, a FAISS vector index, and a BM25 lexical index. The two ranking streams are combined using reciprocal rank fusion (RRF).

Runtime services include workflow state storage, a review permission layer, and a structured JSONL audit log. The final submission package also contains the RAG assets required to reproduce the retrieval workflow.

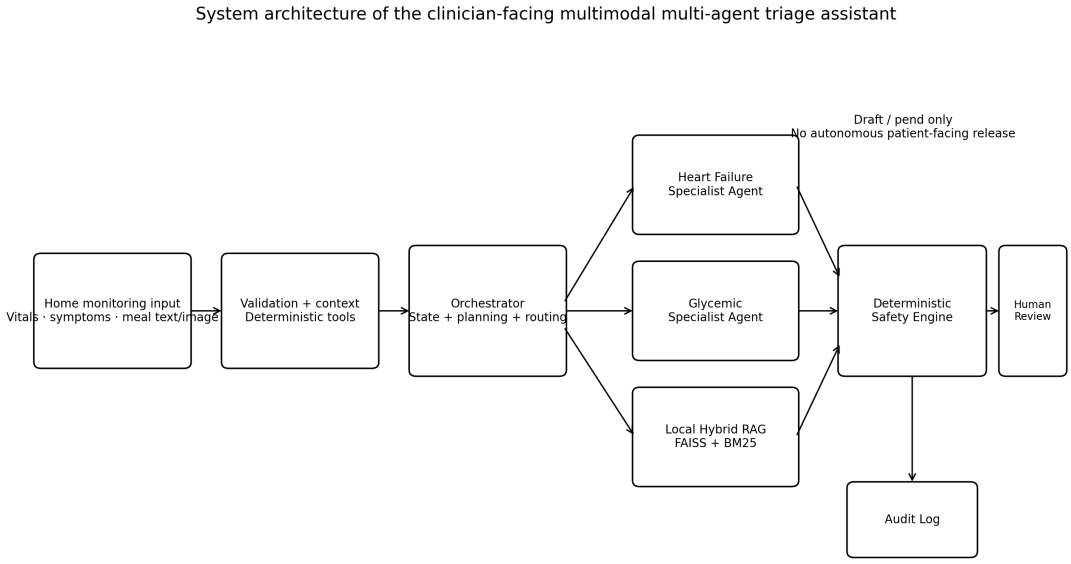


Figure 1. System architecture with the orchestrator, two specialist agents, retrieval, safety gate, review, and audit.

2.2 Agentic workflow through Perceive, Reason, Act, and Human Oversight

The prototype implements an agentic workflow through explicit state transitions rather than a single prompt-response interaction. During Perceive, the system accepts the case, validates required data, runs deterministic tools, and retrieves guideline evidence. During Reason, the Heart Failure and Glycemic specialist components create structured, evidence-linked interpretations. During Act, the system does not execute a clinical intervention. The safety gate makes a workflow-level routing decision, either continuing to clinician review or entering SAFE_STOP. Human Oversight then allows the clinician to approve, edit, reject, or escalate within the permissions allowed by the workflow state.

This definition is deliberately narrower than a consumer interpretation of an “agent” that acts autonomously in the world. In healthcare, action authority has clinical consequences. The design therefore separates agentic coordination from clinical commitment. The agent can plan, retrieve, reason, use tools, and update state, but the release boundary remains human-controlled.

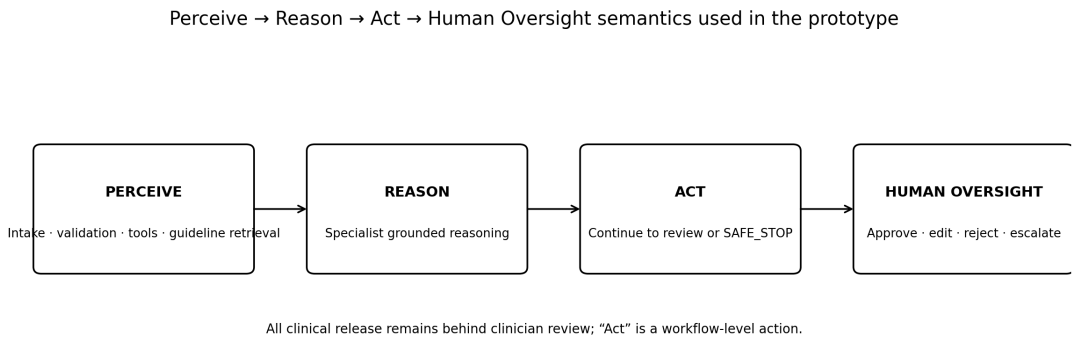


Figure 2. Observable Perceive–Reason–Act loop, ending at a mandatory human checkpoint.

2.3 Specialist agents and deterministic tools

The Heart Failure and Glycemic specialist agents provide domain separation. This allows retrieval queries and structured interpretations to be tailored to the relevant evidence source while preserving a common orchestration contract. The agents consume the same patient state but retrieve different guideline evidence and return structured reasoning with required fields such as reasoning status, patient findings, guideline interpretation, clinical concerns, evidence references, uncertainties, urgency, and a mandatory human-review flag.

Several operations that do not require generative reasoning are intentionally implemented as deterministic tools. These include calculation of weight change, required-data checks, prompt-injection detection, and a conservative multimodal contradiction check. Moving these checks outside the language model improves predictability and testability. For example, the multimodal contradiction tool does not attempt to infer the nutritional content or clinical significance of a meal image. It only detects a narrow machine-checkable contradiction in which patient text explicitly says that no meal was consumed while an image description indicates visible food.

2.4 Retrieval-augmented grounding

Grounding is implemented locally using two domain-specific corpora. The Heart Failure retriever contains 320 chunks derived from the 2026 ESC Guidelines for the management of heart failure [7]. The Glycemic retriever contains 50 chunks from the WHO HEARTS-D material on diagnosis and management of type 2 diabetes [8]. Each chunk preserves provenance metadata including source, page, and chunk

identifier. Precomputed sentence-transformer embeddings are indexed using FAISS, while BM25 provides a lexical retrieval signal. At query time, dense and BM25 candidates are fused using reciprocal rank fusion and the top evidence is supplied to the reasoning component.

The design choice for hybrid retrieval reflects the complementary strengths of semantic and lexical search. Dense retrieval can match conceptually similar passages even when wording differs, whereas BM25 can favour exact terminology that may be important in clinical guidelines. The system does not treat retrieval itself as proof of correctness. Reasoning output is validated against the set of retrieved chunk identifiers and page numbers so that generated evidence references cannot point to a source that was not actually supplied to the model.

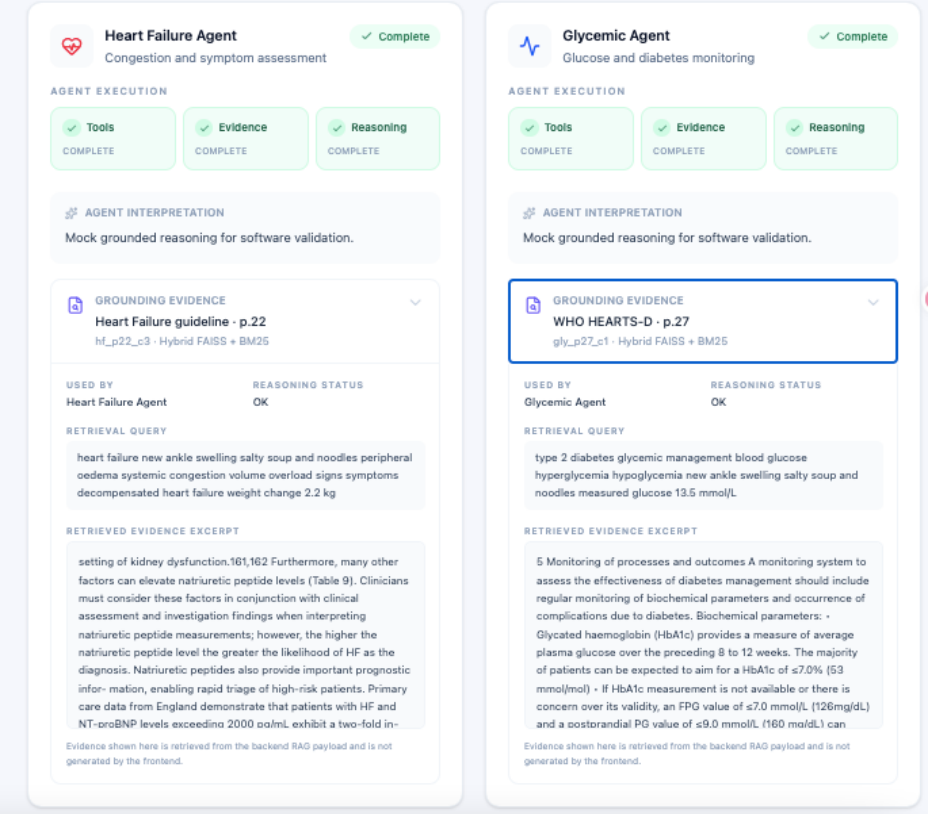


Figure 3. RAG evidence panel showing source, page and retrieved chunk identifiers.

2.5 Prompt-as-specification and machine-checkable output

The generative reasoning layer is treated as a constrained component rather than an unrestricted conversational model. The system prompt specifies that only supplied patient facts, deterministic tool outputs, and retrieved evidence may be used. Patient text and retrieved documents are explicitly marked as data rather than instructions.

The prompt prohibits invented diagnoses, medications, treatment changes, thresholds, missing facts, causal relationships, and citations. It also requires insufficient evidence to be represented explicitly rather than filled in by plausible completion.

The response contract requires valid JSON with a known schema version. A deterministic validator checks required keys, allowed reasoning statuses, allowed urgency labels, the human-review flag, and evidence references. Invalid evidence references or malformed output are not silently accepted. This design follows the course principle that prompts should operate as specifications and that important outputs should be machine-checkable rather than relying on free-text interpretation [1,4].

2.6 Model choice, cost control, and simulation mode

Live specialist reasoning uses a cache-first Gemini client whose default model identifier is gemini-3.5-flash-lite in the verified backend. API use requires an explicit enablement flag and a call budget. The client has no application-level retry. Responses are validated against the structured schema and referenced RAG chunk identifiers. Provider failures become explicit error statuses, which cause fail-closed routing. Caching keys incorporate the model, prompts and output schema to prevent uncontrolled reuse of an incompatible response.

For formal assessment and regression testing, the recommended execution mode is simulation. Simulation uses deterministic mock reasoning while preserving the same orchestration, retrieval, safety, review, and audit structure. This separation provides two benefits. First, it makes the demonstration reproducible and avoids dependency on external API availability. Second, it allows safety-control behaviour to be evaluated without confounding results with model nondeterminism. The formal evaluation reported in Section 4 therefore uses simulation and deterministic components and makes zero live Gemini API calls.

2.7 State management, orchestration, and observability

A key agentic property of the prototype is that the workflow is represented as explicit state rather than as a hidden conversational exchange. The orchestrator assigns a run identifier, emits progress events, invokes specialist components, aggregates outputs, applies the safety gate, drafts a workflow action, and finishes at a human checkpoint.

The frontend receives backend-derived state and renders both the current workflow state and a chronological execution trace. In a standard simulation, the interface shows planning, intake, deterministic tools, guideline retrieval, specialist reasoning, the safety gate, and the human checkpoint rather than inferring these stages from frontend animation.

The prototype keeps runtime state in lightweight in-memory stores suitable for a proof of concept. This design is sufficient to demonstrate waits for human review and to retrieve audit and review state by run identifier, but it is not intended for production concurrency or recovery. A deployed system would require durable transactional storage, idempotent task handling, and explicit recovery semantics for partially completed tool calls or external service failures.

3. Workflow, Safety, and Governance

3.1 Human-defined goal, guardrails, and stop condition

The human-defined goal is to structure multimodal home-monitoring data, retrieve relevant evidence, and prepare a traceable triage case for clinician review. The goal is bounded to clinician-facing support and synthetic H@H prototype data. The interface makes the boundary explicit through a Goal, Guardrails & Authority panel and states that controls are enforced outside the model.

The stop condition is equally explicit. If the Safety Engine returns any safety reason, the workflow enters SAFE_STOP and autonomous continuation is withheld. Safety reasons include missing required data, prototype-stale data, prompt-injection detection, multimodal contradiction, unavailable evidence, model failure, missing specialist output, or a major disagreement in specialist urgency. A normal case with no safety reasons is not “approved” by the AI. It is routed to CLINICIAN_REVIEW.

3.2 Action authority and least-privilege design

The course materials distinguish lower-risk read/derive actions from higher-risk write/commit actions and emphasize separate permissions for draft, pend, approve, and commit [4]. The prototype applies this principle by capping automated authority at T2, which permits drafting and pending review only. T4 autonomous patient-facing commitment is blocked. This is a deliberate least-privilege choice. The agent

may prepare an interpretation and workflow draft, but it cannot convert that output directly into a patient-facing clinical action.

The distinction also reduces ambiguity in the meaning of the agentic "Act" stage. In this project, Act means routing the workflow to clinician review or stopping it safely. It does not mean autonomously sending advice, changing medication, scheduling a clinical intervention, or writing an authoritative clinical record.

3.3 Deterministic safety controls

The safety architecture is intentionally layered. Data quality is checked before reliance on generative reasoning. Prompt-injection patterns in patient-supplied text are detected deterministically. A conservative multimodal contradiction check flags a narrow inconsistency between meal text and the supplied image description. RAG status and model status are inspected so that unavailable or insufficient evidence does not silently propagate. Finally, specialist urgency outputs are compared. A large disagreement causes a SAFE_STOP because the system should not arbitrarily choose one agent over another.

This separation is important because guardrails implemented only as natural-language instructions can fail in the same model that is being constrained. In the prototype, the generative model cannot override the safety engine. The review layer also independently enforces the SAFE_STOP permission boundary, so an Approve or Edit decision raises a permission error rather than changing the final state. This defense-in-depth structure makes critical controls independently testable.

3.4 Human checkpoint and override protection

Every workflow requires clinician review. For a non-SAFE_STOP case, the clinician can Approve, Edit, Reject, or Escalate. For SAFE_STOP, Approve and Edit are disabled by the interface and blocked again by backend permission logic. Reject and Escalate remain available. The two-layer implementation avoids treating a disabled button as the security boundary. Even if a client attempted to submit an invalid review action directly to the API, the backend would reject it.

This design addresses automation bias by keeping professional judgment explicit and by exposing the underlying evidence and safety state next to the decision point. The

course materials note that model performance is not system performance and that human interaction, reliance, and workflow influence real clinical outcomes [3]. The prototype therefore prioritizes recognition and verification. The clinician can inspect the retrieved source, current case data, specialist outputs, and safety reasons before recording a decision.

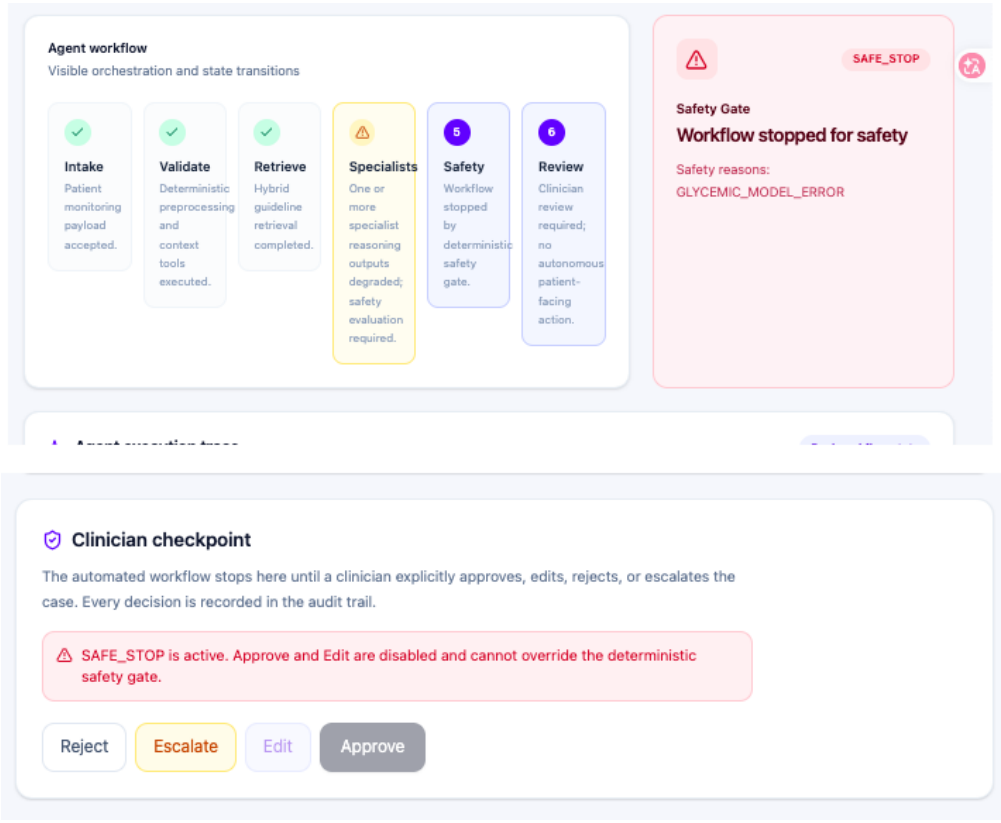


Figure 4. SAFE_STOP screen with Approve and Edit disabled while escalation remains possible.

3.5 Risk register and mitigations

Table 1. Principal risks, implemented controls, and remaining limits.

Risk	Control in prototype	Residual limit
Missing/stale data	Fail-closed safety reason and freshness boundary test	24-hour cut-off is a demonstration policy
Prompt injection / unsupported output	Deterministic detectors, JSON schema checks, and evidence-reference checks	Rule coverage and clinical factuality remain unvalidated
Specialist conflict / model failure	SAFE_STOP on major disagreement or dependency error	Not evaluated on real patient cohorts
Automation bias / unauthorised action	Mandatory clinician review with backend blocking of SAFE_STOP Approve and Edit	Requires prospective usability and access-control testing

3.6 Auditability and governance

The audit contract records the fields required by the project brief. These are input, action, source, timestamp, output, and human decision. Triage events capture a patient snapshot, execution mode/provider, RAG evidence provenance, deterministic safety source, and the final workflow/safety state. Review events capture the attempted or accepted clinician action, the permission layer, the review output, and the human decision. Blocked attempts are also auditable, which is particularly important for demonstrating that SAFE_STOP is enforced rather than merely displayed.

This structure supports accountability by making the workflow reconstructable after the fact. The course materials emphasize intended use, documented limitations, validation before release, audit trails, and the ability to monitor or switch off deployed systems [5]. The prototype addresses these principles at proof-of-concept level through narrow intended use, explicit limitations, version-controlled source code, structured evaluation, and auditable workflow events. It does not claim production compliance. Authentication, role-based access control, institutional data governance, incident management, and post-market monitoring remain future-work requirements.

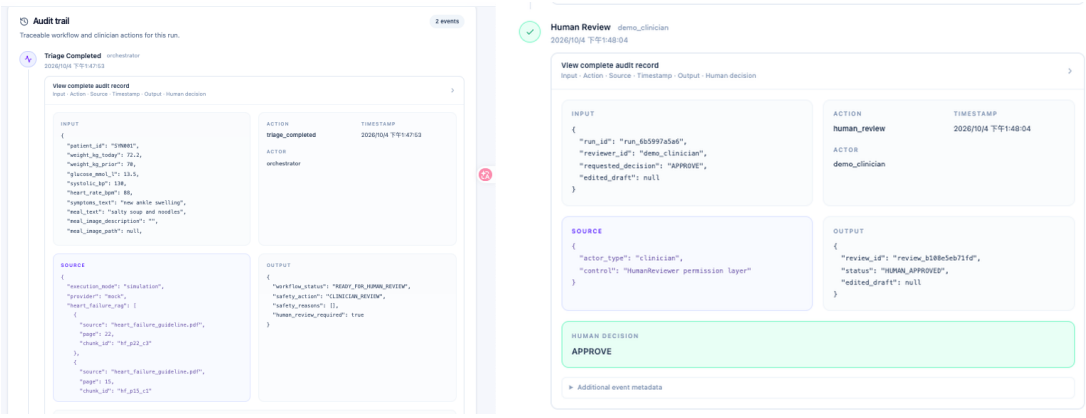


Figure 5. Structured audit record containing input, action, source, timestamp, output, and human decision.

3.7 Governance alignment and deployment boundary

The governance boundary is defined by intended use. The system is a clinician-facing prototype that prepares an evidence-linked triage package for review. It is not presented as an autonomous diagnostic or treatment system. This distinction matters because governance obligations and risk increase when a system changes from

drafting or summarising to committing a clinical action. The project therefore treats the action boundary as part of the technical architecture rather than as a disclaimer added after implementation.

Important deployer responsibilities remain unimplemented. A real healthcare institution would need to determine whether the tool fits its patient population and clinical workflow, establish local acceptance criteria, train users, manage access and data retention, monitor incidents and drift, and maintain a mechanism to suspend the system. These are intentionally stated as future requirements rather than implied by the prototype.

4. Prototype, Demo, and Evaluation

4.1 Prototype interface and glass-box observability

The prototype interface is designed as a clinician workspace rather than as an agent animation. The primary layer presents the patient case, goal and guardrails, clinical review summary, specialist findings, evidence, safety status, and human decision controls. The observability layer exposes how the system reached that state through backend-derived execution steps, state transitions, deterministic tool use, RAG provenance, specialist reasoning status, safety evaluation, and audit records.

This two-layer design resolves a tension in agentic demonstrations. A purely operational dashboard may hide the agentic behaviour required for assessment, while a trace-heavy interface may feel like watching a debugging console rather than using a clinical tool. The final interface therefore makes the clinician task primary and uses expandable glass-box details as evidence and verification.

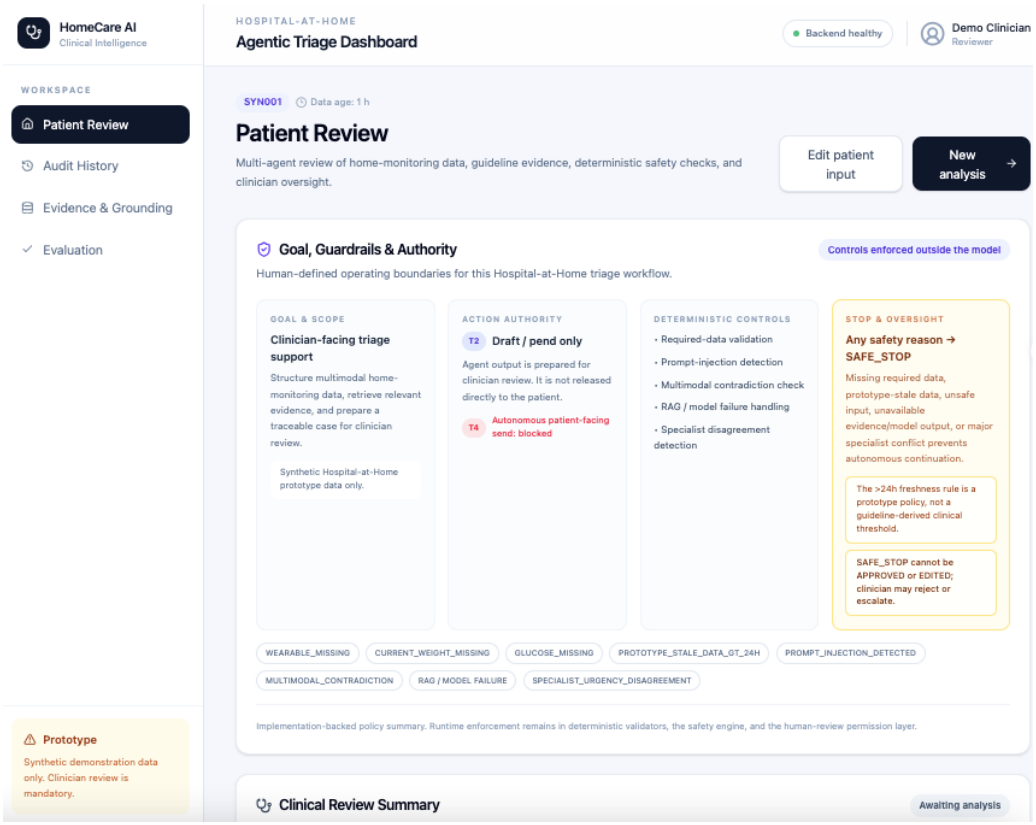


Figure 6. Clinician-facing patient-review workspace and exposed guardrails.

4.2 Demonstration scenarios

Three demonstration scenarios are available. Standard demonstrates the normal path from input through specialist analysis and grounding to CLINICIAN_REVIEW. SAFE_STOP Demo intentionally triggers deterministic safety handling so that the blocked workflow and human review constraints can be shown. Provider Failure demonstrates degraded external-model behaviour without requiring a live API call. The failed specialist path is converted into a structured model-error status and then into SAFE_STOP.

The scenarios are not presented as clinical vignettes with validated outcomes. They are software-validation cases designed to show whether the workflow follows its declared policies. Synthetic data are used throughout the formal evaluation and recommended demonstration mode.

4.3 Formal evaluation design

The formal evaluation suite contains twenty-three test cases. The original fifteen cases cover eight end-to-end safety paths, two dependency failures, one major multi-

agent urgency conflict, and four human-oversight permission cases. Eight additional robustness and reasoning-validation cases test the exact 24-hour freshness boundary, paraphrased English and Chinese prompt-injection attempts, a one-level urgency difference that should not over-trigger the conflict rule, malformed model JSON, fabricated evidence references, an attempt by model output to disable human review, and a clinical concern returned without evidence. The suite executes in simulation and deterministic mode and reports zero live Gemini API calls, making the expected control behaviour reproducible without provider nondeterminism.

The evaluation deliberately includes both positive and negative cases. The positive path verifies that a normal case reaches CLINICIAN_REVIEW, that a non-SAFE_STOP workflow can be approved, that exactly 24 hours does not trigger the prototype stale-data rule, and that a one-level specialist urgency difference does not produce a major-conflict stop. Negative cases verify that missing data, data older than the prototype freshness policy, prompt injection, multimodal contradiction, unavailable RAG, model failure, strong specialist disagreement, malformed model JSON, invalid citations, missing citations, and an invalid attempt to disable human review fail closed. Permission tests confirm that SAFE_STOP cannot be approved or edited while escalation remains available.

Table 2. Formal evaluation coverage, grouped by behaviour.

Category	Examples	Outcome
Input and safety checks	Missing/stale wearable, weight/glucose, multimodal contradiction, injection	11/11 passed
Dependency and reasoning	RAG/model failure, malformed JSON, missing/fabricated citation, review-flag override	6/6 passed
Specialist disagreement	Major and one-level urgency differences	2/2 passed
Human oversight	SAFE_STOP blocks Approve and Edit, allows Escalate, and permits Approve in the normal workflow	4/4 passed
Total	Synthetic, simulation-mode prototype-conformance tests	23/23 passed with zero live calls

4.4 Evaluation results and interpretation

All twenty-three tests passed, giving a 100% pass rate on the verified prototype-conformance suite. This means that the tested software controls behaved as specified for the supplied synthetic scenarios. In particular, the suite demonstrates end-to-end fail-closed behaviour, dependency-failure handling, cross-agent conflict handling,

human-override restrictions, boundary-condition handling, reasoning-schema validation, evidence-reference validation, and enforcement of the human-review invariant. The result also confirms that these safety behaviours can be evaluated without live model calls.

The 100% pass rate must not be interpreted as clinical accuracy or real-world safety. The test set is synthetic and targeted at declared workflow requirements. It does not measure sensitivity or specificity for clinical deterioration, calibration, subgroup performance, downstream patient outcomes, or clinician workload reduction. The reasoning/citation tests verify machine-checkable invariants such as valid JSON, evidence-reference allow-listing, and mandatory human review. They do not establish that a model-generated clinical statement is medically correct. The course materials identify task completion, unsupported-claim rate, source/citation accuracy, retrieval precision/recall, latency, and calibration as relevant evaluation dimensions for agentic clinical workflows [4]. This project adds a small frozen retrieval benchmark for precision/recall-style evidence while retaining clinical factuality, calibration, and human-factors testing as future work.

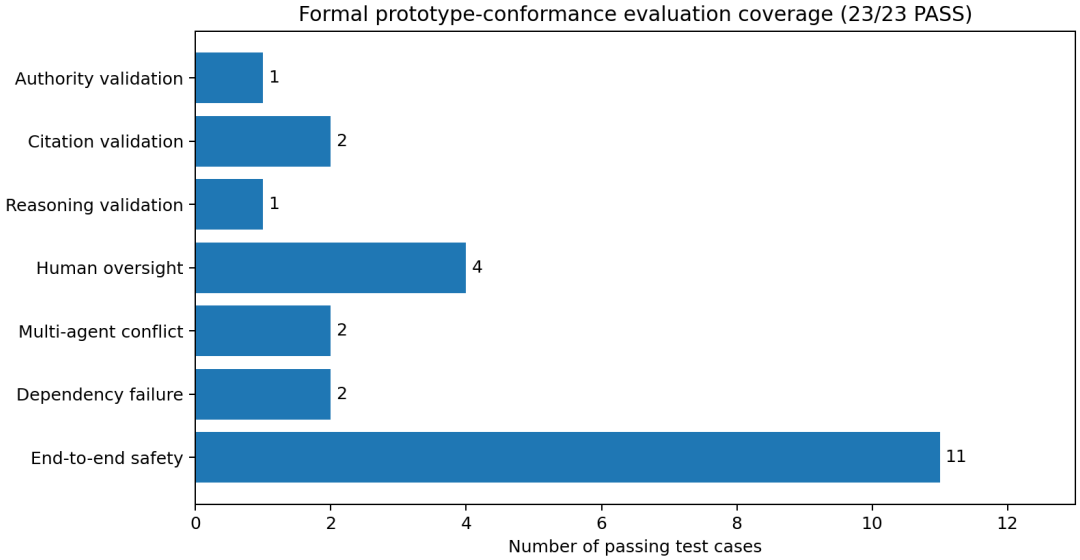


Figure 7. Simulation test coverage across the 23 prototype-conformance cases.

A manually source-verified 10-query retrieval benchmark was frozen before metric computation to reduce the risk of post-hoc relevance labelling. With k=3 (meaning only the top three retrieved chunks are considered for each query), the production equal-weight FAISS+BM25 RRF retriever achieved macro Precision@3 = 0.3666,

Recall@3 = 0.6500, Hit Rate@3 = 0.9000, and MRR@3 = 0.7333. Glycemic retrieval was stronger than Heart Failure retrieval on this small benchmark. Candidate-depth and weighted-RRF ablations were evaluated but not adopted. Larger candidate pools reduced overall Hit Rate@3, while BM25-heavy fusion recovered the HF-Q2 failure at the cost of regressions on other queries. The production equal-weight RRF configuration was therefore retained to avoid tuning directly to the same small evaluation set.

Table 3. Frozen RAG retrieval metrics for the three highest-ranked results across ten source-verified queries.

Benchmark	P@3 (macro)	R@3 (macro)	Hit@3	MRR@3
10 frozen queries	0.367	0.650	0.900	0.733

In a separate live integration smoke test, one synthetic case obtained two uncached Gemini 3.5 Flash-Lite specialist responses (HF and Glycemic), both accepted by schema checks. The workflow remained CLINICIAN_REVIEW with human review required. This does not extend the 23/23 simulation result to live-model robustness or real-world safety.

4.5 Reproducibility and submission package

The submitted prototype archive contains backend and frontend source code, Python requirements, indexed RAG assets for both specialties, scripted workflow tests, frozen retrieval labels, and recorded result files. Runtime credentials, local caches and sensitive data should not be included. The fresh environment may need to retrieve the configured sentence-embedding model before using the FAISS indices.

The integration gate recorded successful installation of Python requirements, a fresh backend and both RAG asset sets, 23-case simulated regression, frozen retrieval benchmark, frontend npm ci and production build, frontend/backend health checks, and evaluation API proxy. A second simulation workflow gate passed normal review, execution trace, guideline grounding, clinician approval, SAFE_STOP restrictions, escalation, and the six-field audit contract. These checks are software integration evidence, not evidence of clinical performance.

4.6 Evidence mapping to prototype requirements

The prototype was intentionally designed so that each required demonstration artefact is visible in the interface rather than existing only in code. User input is

represented by the patient review form and scenario selector. Agent workflow is represented by backend-derived progress, stage transitions, and the Perceive–Reason–Act–Human Oversight trace. Grounding is represented by expandable source/page/chunk evidence. Human oversight is represented by approve, edit, reject, and escalate controls. Safety is represented by explicit SAFE_STOP reasons and blocked decisions. Auditability is represented by complete event records. Evaluation artefacts are exposed through a dedicated workspace backed by the submitted result JSON.

This design is useful beyond assessment alignment because it creates a single evidence surface for debugging and governance review. The clinician sees the operational summary first, while a technical or governance reviewer can inspect deeper provenance and state when needed. The interface therefore acts as both a task workspace and an observability layer.

Implementation evidence includes workflow screenshots (Figures 1–6), recorded review and audit decisions, 23 scripted cases, and the frozen 10-query retrieval benchmark.

5. Discussion, Future Work, and Conclusion

5.1 What the prototype demonstrates

The main contribution of this project is not a new clinical prediction model. It is an implementation pattern for bounded agentic workflow in a high-stakes setting. The prototype shows that planning, retrieval, specialist reasoning, safety logic, human oversight, and auditability can be separated into explicit components with independently testable contracts. This separation makes the system easier to inspect and safer to demonstrate than a single end-to-end prompt that directly emits a clinical recommendation.

The project also illustrates why agentic AI in healthcare should be evaluated as a system rather than as a language model. The behaviour that matters to the user includes whether required data are present, whether evidence can be retrieved, whether two specialists disagree, whether a dependency fails, whether the workflow stops correctly, whether a human decision is allowed, and whether the event can be

reconstructed later. These are workflow properties, not merely text-generation properties.

5.2 Design evolution from proposal to final prototype

The early proposal described a broader patient-intervention concept and named a different implementation stack. During prototyping, the project was narrowed to a clinician-facing triage assistant and implemented using React/TypeScript and FastAPI. This change is a design refinement rather than a loss of agentic functionality. The final system retains multi-agent orchestration, tools, local RAG, state transitions, safety controls, human review, and auditability while reducing the system's authority to act on patients.

5.3 Limitations

First, the formal evaluation is a prototype-conformance evaluation, not a clinical validation study. The twenty-three cases are synthetic and targeted at known control paths. Mechanical reasoning/citation validators are tested, but no clinician-annotated benchmark was used to estimate clinical factuality, sensitivity to deterioration events, calibration, subgroup performance, or clinical utility. Second, the multimodal image path is limited. The final safety logic can reason over an image description and can detect a narrow contradiction, but the project does not validate image-based dietary interpretation as a clinical measurement.

Third, the RAG corpora are limited to two guideline sources. Retrieval was evaluated on a manually source-verified 10-query frozen benchmark, yielding macro Precision@3 = 0.3666, Recall@3 = 0.6500, Hit Rate@3 = 0.9000, and MRR@3 = 0.7333. The benchmark exposed a Heart Failure retrieval failure (HF-Q2) and showed that candidate-depth or BM25-heavy tuning can trade one failure for regressions elsewhere. Because the benchmark is small and was used for exploratory ablation, it is not sufficient for production tuning. A larger independently constructed held-out set is needed. Fourth, live provider reliability was not established as part of the formal evaluation. Development testing observed provider failure in the glycemic path. The architecture intentionally converts such failure into a structured SAFE_STOP rather than retrying silently.

Fifth, production security and governance are out of scope. The prototype does not implement institutional authentication, SSO/OIDC, role-based access control, production databases, encrypted clinical interfaces, incident management, or data-retention policies. Finally, alert-fatigue reduction is an intended workflow benefit rather than a measured outcome. Demonstrating actual workload reduction would require prospective user testing with clinicians in a representative operational setting.

5.4 Future work

Future priorities are clinician-labelled safety evaluation, prospective usability and workload measurement, subgroup and retrieval validation, durable workflow state, role-based access control, incident reporting, and data-drift monitoring.

5.5 Conclusion

This project developed a clinician-facing multimodal multi-agent triage assistant for Hospital-at-Home chronic disease monitoring. The prototype integrates explicit planning and state transitions, deterministic tools, two specialist agents, local hybrid RAG, structured reasoning, a deterministic Safety Engine, human review permissions, and a complete audit trail. The interface exposes both the clinical task layer and the agent observability layer, satisfying the requirement that the workflow be visible rather than hidden behind a chatbot-style answer.

The most important design choice is the authority boundary. The system may structure information and draft a case, but it cannot autonomously release a clinical recommendation. Any safety reason causes SAFE_STOP. Model and RAG failures also fail closed, and strong specialist disagreement is escalated. A case in SAFE_STOP cannot be approved or edited. A twenty-three-case simulation/deterministic evaluation verified these prototype controls under the tested scenarios with zero live Gemini API calls. A separate frozen 10-query retrieval benchmark quantified the evidence-retrieval layer and exposed a real failure case rather than tuning it away.

The prototype therefore supports the feasibility of a bounded, evidence-linked, human-supervised agentic workflow. It does not establish clinical efficacy or deployment readiness. Future work should focus on clinician-labelled evaluation,

retrieval and citation quality, human-factors testing, production security, standards-based data integration, and lifecycle governance before any real clinical use.

References

[1] NTU CA6117 AI in Healthcare. Course Project Brief — Agentic AI in Healthcare. Project brief, deliverables, assessment criteria, and prototype expectations. 2026.

[2] NTU CA6117 AI in Healthcare. Lecture 5b — Hospital at Home / Care at Home. Moving acute care out of the ward. 2026.

[3] NTU CA6117 AI in Healthcare. Lecture 5a — Human-AI Interaction (HAX). 2026.

[4] NTU CA6117 AI in Healthcare. Lecture 4b — Agentic AI and Clinical Workflow. 2026.

[5] NTU CA6117 AI in Healthcare. Lecture 7b — Governance. 2026.

[6] Wong A, Otles E, Donnelly JP, et al. External Validation of a Widely Implemented Proprietary Sepsis Prediction Model in Hospitalized Patients. JAMA Internal Medicine. 2021. 181(8), 1065–1070.

[7] European Society of Cardiology. 2026 ESC Guidelines for the management of heart failure. European Heart Journal. 2026, advance publication. DOI 10.1093/eurheartj/ehag100.

[8] World Health Organization. HEARTS D — Diagnosis and management of type 2 diabetes. HEARTS technical package for cardiovascular disease management in primary health care. Geneva, World Health Organization, 2020. WHO/UCN/NCD/20.1.

[9] CA6117 Group 14. Project proposal — Multimodal multi-agent triage workflow for hospital at home chronic disease management. 2026.

Appendix A. Detailed Prototype-Conformance Test Cases

Full per-case expected/actual outcomes and reasons are available in evaluation/results/evaluation_summary.md and evaluation_results.json. The table below lists test scenarios and pass status.

Table A1. Formal regression suite covering 23 synthetic test cases in simulation mode.

ID	Test scenario	Result
EVAL-01	Standard monitoring case	PASS
EVAL-02	Wearable data missing	PASS
EVAL-03	Monitoring data older than prototype freshness policy	PASS
EVAL-04	Current weight missing	PASS
EVAL-05	Glucose reading missing	PASS
EVAL-06	Prompt-injection attempt in patient-supplied text	PASS
EVAL-07	Meal text contradicts meal-image description	PASS
EVAL-08	Multiple simultaneous input safety issues	PASS
EVAL-16	Exact 24-hour freshness boundary	PASS
EVAL-17	Paraphrased English prompt injection	PASS
EVAL-18	Chinese prompt injection	PASS
EVAL-09	Heart Failure RAG unavailable	PASS
EVAL-10	Glycemic reasoning model failure	PASS
EVAL-11	Specialists disagree strongly on urgency	PASS
EVAL-12	Clinician attempts APPROVE on SAFE_STOP	PASS
EVAL-13	Clinician attempts EDIT on SAFE_STOP	PASS
EVAL-14	Clinician escalates SAFE_STOP	PASS
EVAL-15	Clinician approves non-SAFE_STOP workflow	PASS
EVAL-19	One-level urgency difference	PASS
EVAL-20	Model returns malformed JSON	PASS
EVAL-21	Model fabricates a chunk citation not present in retrieved evidence	PASS
EVAL-22	Model attempts to set needs_human_review to false	PASS
EVAL-23	Clinical concern is returned without an evidence citation	PASS

Appendix B. Generative AI Use Declaration

I used Google Gemini to support code review, debugging, test-case design in colab and OpenAI ChatGPT to optimize the report translation. I checked AI-generated suggestions against the implementation, course materials, guideline sources, and available prototype test evidence before including them. I remain responsible for the design, code, analysis, interpretation, and final submission. I did not provide real patient data or confidential clinical records to the AI tool. The prototype and its evaluation use synthetic demonstration data.